

Temel Örnekler

Django, geniş bir uygulama yelpazesine hitap edecek temel görünüm (view) sınıfları sunar. Tüm görünümler; görünümün URL'lere bağlanması, HTTP metotlarının yönlendirilmesi (dispatching) ve diğer ortak özellikleri yöneten **View** sınıfından türetilir. **RedirectView** bir HTTP yönlendirmesi sağlar; **TemplateView** ise bir şablonu (template) ekrana basmak (render etmek) için temel sınıfı genişletir.

URLconf İçinde Kullanım

Genel görünümleri (generic views) kullanmanın en doğrudan yolu, onları doğrudan URLconf dosyanızda oluşturmaktır. Eğer sınıf tabanlı bir görünümün (class-based view) sadece birkaç özneliğini değiştiriyorsanız, bunları **as_view()** metod çağrısının içine parametre olarak geçebilirsiniz:

```
from django.urls import path
from django.views.generic import TemplateView

urlpatterns = [
    path("about/", TemplateView.as_view(template_name="about.html")),
]
```

as_view() metoduna iletilen her türlü argüman, sınıfta tanımlanmış olan özneliklerin (attributes) üzerine yazılır. Bu örnekte, **TemplateView** üzerindeki **template_name** özneliğini ayarladık. Benzer bir üzerine yazma (overriding) deseni, **RedirectView** üzerindeki **url** özneliği için de kullanılabilir.

Genel Görünümleri Alt Sınıfa Ayırma (Subclassing)

Genel görünümleri kullanmanın ikinci ve daha güçlü yolu, mevcut bir görünümünden miras almak ve alt sınıfınızda yeni değerler veya metotlar sağlamak için öznelikleri (örneğin **template_name**) veya metotları (örneğin **get_context_data**) geçersiz kılmaktır (override). Örnek olarak, yalnızca **about.html** şablonunu görüntüleyen bir görünümü ele alalım. Django'nun bunu yapmak için genel bir görünümü (**TemplateView**) vardır; bu yüzden onu alt sınıfa ayırabilir ve şablon adını geçersiz kılabiliriz:

```
# some_app/views.py
from django.views.generic import TemplateView

class AboutView(TemplateView):
    template_name = "about.html"
```

Ardından, bu yeni görünümü URLconf dosyamıza eklememiz gerekir. `TemplateView` bir fonksiyon değil, bir sınıftır; bu nedenle URL'yi sınıfın `as_view()` metoduna yönlendiririz. Bu metot, sınıf tabanlı görünümlere fonksiyon benzeri bir giriş noktası sağlar:

```
# urls.py
from django.urls import path
from some_app.views import AboutView

urlpatterns = [
    path("about/", AboutView.as_view()),
]
```

Yerleşik genel görünümlerin nasıl kullanılacağı hakkında daha fazla bilgi için bir sonraki konu olan **genel sınıf tabanlı görünümler (generic class-based views)** kısmına bakınız.

Diğer HTTP Metotlarını Desteklemek

Birinin, görünümleri bir API olarak kullanarak kitaplığımıza HTTP üzerinden erişmek istediğini varsayalım. API istemcisi ara sıra bağlanacak ve son ziyaretten bu yana yayınlanan kitapların verilerini indirecektir. Ancak o zamandan beri yeni bir kitap eklenmemişse, veritabanından kitapları çekmek, tam bir yanıt oluşturmak ve bunu istemciye göndermek CPU süresi ve bant genişliği israfıdır. API'ye en son kitabın ne zaman yayınlandığını sormak daha tercih edilebilir bir yöntem olabilir.

URL'yi URLconf içinde kitap listesi görünümüne eşliyoruz:

```
from django.urls import path
from books.views import BookListView

urlpatterns = [
    path("books/", BookListView.as_view()),
]
```

Görünüm ise şu şekildedir:

```
from django.http import HttpResponse
from django.views.generic import ListView
from books.models import Book

class BookListView(ListView):
    model = Book

    def head(self, *args, **kwargs):
        last_book = self.get_queryset().latest("publication_date")
        response = HttpResponse(
            # RFC 1123 date format.
            headers={
                "Last-Modified": last_book.publication_date.strftime(
                    "%a, %d %b %Y %H:%M:%S GMT"
                )
            },
        )
        return response
```

Görünüme bir `GET` isteği ile erişilirse, yanıtta bir nesne listesi döndürülür (`book_list.html` şablonu kullanılarak). Ancak istemci bir `HEAD` isteği gönderirse, yanıtın gövdesi boş olur ve `Last-Modified` başlığı en son kitabın ne zaman yayınlandığını belirtir. İstemci bu bilgiye dayanarak nesne listesinin tamamını indirip indirmemeye karar verebilir.

Asenkron Sınıf Tabanlı Görünümler

Daha önce gösterilen senkron (`def`) metot işleyicilerinin yanı sıra, `View` alt sınıfları `await` kullanarak asenkron koddan yararlanmak için asenkron (`async def`) metot işleyicileri de tanımlayabilir:

```
import asyncio
from django.http import HttpResponse
from django.views import View

class AsyncView(View):
    async def get(self, request, *args, **kwargs):
        # Perform io-blocking view logic using await, sleep for example.
        await asyncio.sleep(1)
        return HttpResponse("Hello async world!")
```

Tek bir görünüm sınıfı içinde, kullanıcı tarafından tanımlanan tüm metot işleyicileri ya `def` kullanılarak tamamen senkron, ya da `async def` kullanılarak tamamen asenkron olmalıdır. Eğer `def` ve `async def` tanımlamaları karıştırılırsa, `as_view()` içinde bir `ImproperlyConfigured` istisnası (exception) fırlatılacaktır.

Django, asenkron görünümleri otomatik olarak algılayacak ve bunları asenkron bir bağlamda çalıştıracaktır. Django'nun asenkron desteği ve asenkron görünümlerin en iyi şekilde nasıl kullanılacağı hakkında daha fazla bilgiyi **Asynchronous support** (Asenkron destek) bölümünde okuyabilirsiniz.